

Задача 1

Создайте ваш первый Docker Swarm-кластер в Яндекс Облаке. Документация swarm: https://docs.docker.com/engine/reference/commandline/swarm_init/

1. Создайте 3 облачные виртуальные машины в одной сети.
2. Установите docker на каждую VM.
3. Создайте swarm-кластер из 1 мастера и 2-х рабочих нод.
4. Проверьте список нод командой:

```
1 docker node ls
```

Задача 2 () (необязательное задание).

1. Задеплойте ваш python-fork из предыдущего ДЗ(05-virt-04-docker-in-practice) в получившийся кластер.
2. Удалите стенд.

Детальный разбор задачи:

Синтаксис команд для развертывания VM из консоли `yc`: официальная документация: cloud.yandex.ru/docs/cli

Для единоразовой установки 3 VM создал скрипт:

```
1  #!/bin/bash
2
3  echo "Create swarm-manager"
4  yc compute instance create --name swarm-managere --zone ru-central1-a --network-
   interface subnet-name=default-ru-central1-a,nat-ip-version=ipv4 --create-boot-disk
   image-folder-id=standard-images,image-family=ubuntu-2204-lts,size=20 --memory 2 --
   cores 2 --metadata-from-file user-data=cloud-init.yaml
5
6
7  echo "Create worker-1"
8  yc compute instance create --name swarm-worker-1 --zone ru-central1-a --network-
   interface subnet-name=default-ru-central1-a,nat-ip-version=ipv4 --create-boot-disk
   image-folder-id=standard-images,image-family=ubuntu-2204-lts,size=20 --memory 2 --
   cores 2 --metadata-from-file user-data=cloud-init.yaml
9
10 echo "Create worker-2"
11 yc compute instance create --name swarm-worker-2 --zone ru-central1-a --network-
   interface subnet-name=default-ru-central1-a,nat-ip-version=ipv4 --create-boot-disk
   image-folder-id=standard-images,image-family=ubuntu-2204-lts,size=20 --memory 2 --
   cores 2 --metadata-from-file user-data=cloud-init.yaml
```

```
1  chmod +x create-workers.sh
```

Создал файл `cloud-init.yaml` для автоматического выполнения при первом запуске VM

```
1 #cloud-config
2 users:
3   - name: kva
4     groups: sudo, docker
5     shell: /bin/bash
6     sudo: ['ALL=(ALL) NOPASSWD:ALL'] # настройка прав sudo для пользователя kva
    для учебного стенда, выполнение команд без пароля
7     ssh-authorized-keys:
8       - ssh-ed25519
        AAAAC3NzaC1lZDI1NTE5AAAAICGCxz6Ttx4jaiMvpdANwl4va+6RTIWd+Vd8ylDNu1xD vboxuser@KVA
        # публичный ключ SSH
9
10  package_update: true
11  package_upgrade: true
12
13  packages:
14    - ca-certificates
15    - curl
16    - gnupg
17    - lsb-release
18
19  runcmd: # Установка Docker
```

```
20 - curl -fsSL https://get.docker.com -o /tmp/get-docker.sh # скачал скрипт в
    директорию
21 - sh /tmp/get-docker.sh # запустил
22 - usermod -aG docker kva # добавил пользователя в группу Docker
23 - systemctl enable docker # включил автозапуск
24 - systemctl start docker # запустил Docker
25 - rm /tmp/get-docker.sh # удалил скрипт

1 ./create-workers.sh # запустил скрипт
```

vboxuser@KVA:~/shvird-example-python\$./create-workers.sh

Create swarm-manager

done (49s)

id: fhm40nbf976l6nmvfsir

folder_id: blgrsjmidldjs2rmkgd0

created_at: "2026-05-02T16:00:58Z"

name: swarm-managere

zone_id: ru-central1-a

platform_id: standard-v2

resources:

memory: "2147483648"

cores: "2"

core_fraction: "100"

status: RUNNING

metadata_options:

gce_http_endpoint: ENABLED

aws_v1_http_endpoint: ENABLED

gce_http_token: ENABLED

aws_v1_http_token: DISABLED

aws_v2_http_endpoint: DISABLED

aws_v2_http_token: DISABLED

boot_disk:

mode: READ_WRITE

device_name: fhmsnm849424rrqas266

auto_delete: true

disk_id: fhmsnm849424rrqas266

network_interfaces:

- index: "0"

mac_address: d0:0d:40:5d:6f:49

subnet_id: e9b0lut3pmjp16qqltrq

primary_v4_address:

address: 10.128.0.3

one_to_one_nat:

address: 93.77.176.108

ip_version: IPV4

serial_port_settings:

ssh_authorization: OS_LOGIN

gpu_settings: {}

fqdn: fhm40nbf976l6nmvfsir.auto.internal

scheduling_policy: {}

network_settings:

type: STANDARD

```
placement_policy: {}  
hardware_generation:
```

```
legacy_features:
  pci_topology: PCI_TOPOLOGY_V2
application: {}

Create worker-1...
done (42s)
id: fhm3h1rjakk4dihvjcls
folder_id: blgrsjmidldjs2rmkgd0
created_at: "2026-05-02T16:01:54Z"
name: swarm-worker-1
zone_id: ru-central1-a
platform_id: standard-v2
resources:
  memory: "2147483648"
  cores: "2"
  core_fraction: "100"
status: RUNNING
metadata_options:
  gce_http_endpoint: ENABLED
  aws_v1_http_endpoint: ENABLED
  gce_http_token: ENABLED
  aws_v1_http_token: DISABLED
  aws_v2_http_endpoint: DISABLED
  aws_v2_http_token: DISABLED
boot_disk:
  mode: READ_WRITE
  device_name: fhmp161slfa2g05rvk7l
  auto_delete: true
  disk_id: fhmp161slfa2g05rvk7l
network_interfaces:
  - index: "0"
    mac_address: d0:0d:38:d7:73:55
    subnet_id: e9b0lut3pmjpl6qqltrq
    primary_v4_address:
      address: 10.128.0.24
    one_to_one_nat:
      address: 130.193.51.191
    ip_version: IPV4
serial_port_settings:
  ssh_authorization: OS_LOGIN
gpu_settings: {}
```

```
fqn: fhm3h1rjakk4dihvjcls.auto.internal
scheduling_policy: {}
network_settings:
  type: STANDARD
placement_policy: {}
hardware_generation:
  legacy_features:
    pci_topology: PCI_TOPOLOGY_V2
application: {}
```

Create worker-2...

done (44s)

id: fhmesgqjabc49qc1sdgb

folder_id: blgrsjmidldjs2rmkgd0

created_at: "2026-05-02T16:02:37Z"

name: swarm-worker-2

zone_id: ru-central1-a

platform_id: standard-v2

resources:

memory: "2147483648"

cores: "2"

core_fraction: "100"

status: RUNNING

metadata_options:

gce_http_endpoint: ENABLED

aws_v1_http_endpoint: ENABLED

gce_http_token: ENABLED

aws_v1_http_token: DISABLED

aws_v2_http_endpoint: DISABLED

aws_v2_http_token: DISABLED

boot_disk:

mode: READ_WRITE

device_name: fhmmu7dbr5cjbv76olud

auto_delete: true

disk_id: fhmmu7dbr5cjbv76olud

network_interfaces:

- index: "0"

mac_address: d0:0d:ee:43:53:52

subnet_id: e9b0lut3pmjpl6qqltrq

primary_v4_address:

address: 10.128.0.31


```
one_to_one_nat:
  address: 111.88.246.229
  ip_version: IPV4
serial_port_settings:
  ssh_authorization: OS_LOGIN
gpu_settings: {}
fqdn: fhmesgqjabcd49qclsdgb.auto.internal
scheduling_policy: {}
network_settings:
  type: STANDARD
placement_policy: {}
hardware_generation:
  legacy_features:
    pci_topology: PCI_TOPOLOGY_V2
application: {}
```

- 1 # узнал IP-адреса всех трёх ВМ:
- 2 yc compute instance list

```
• vboxuser@KVA:~/shvird-example-python$ yc compute instance list
```

ID	NAME	ZONE ID	STATUS	EXTERNAL IP	INTERNAL IP
fhm38l3ehdf0efk8edk8	swarm-worker-2	ru-central1-a	RUNNING	62.84.115.133	10.128.0.18
fhm3h1rjakk4dihvjcls	swarm-worker-1	ru-central1-a	RUNNING	130.193.51.191	10.128.0.24
fhm40nbf976l6nmvfsir	swarm-managere	ru-central1-a	RUNNING	93.77.176.108	10.128.0.3

в том числе внутренние IP адреса контейнеров:

- 1 yc compute instance list --format yaml | grep -E "name:|address:"

```

vboxuser@KVA:~/shvird-example-python$ yc compute instance list --format yaml | grep -E "name:|address:"
name: swarm-worker-2
  device_name: fhmja199u2965cgs7j62
  mac_address: d0:0d:34:54:6e:8b
  primary_v4_address:
    address: 10.128.0.18
    address: 62.84.115.133
name: swarm-worker-1
  device_name: fhmp161slfa2g05rvk7l
  mac_address: d0:0d:38:d7:73:55
  primary_v4_address:
    address: 10.128.0.24
    address: 130.193.51.191
name: swarm-managere
  device_name: fhmsnm849424rrqas266
  mac_address: d0:0d:40:5d:6f:49
  primary_v4_address:
    address: 10.128.0.3
    address: 93.77.176.108

```

узнал ID security group:

```
1 yc vpc security-group list
```

```

vboxuser@KVA:~/shvird-example-python$ yc vpc security-group list
+-----+-----+-----+-----+
| ID | NAME | DESCRIPTION | NETWORK-ID |
+-----+-----+-----+-----+
| enpo2auu0766h45ivobi | default-sg-enp0sh2esfudgng27th4 | Default security group for network | enp0sh2esfudgng27th4 |
+-----+-----+-----+-----+

```

узнал CIDR подсети:

```
1 yc vpc subnet get default-ru-central1-a
```

```
• vboxuser@KVA:~/shvird-example-python$ yc vpc subnet get default-ru-central1-a
id: e9b0lut3pmjp16qqltrq
folder_id: blgrsjmidldjs2rmkgd0
created_at: "2025-03-11T08:34:42Z"
name: default-ru-central1-a
description: Auto-created default subnet for zone ru-central1-a in default
network_id: enp0sh2esfudgng27th4
zone_id: ru-central1-a
v4_cidr_blocks:
  - 10.128.0.0/24
```

Открыл необходимые для Docker Swarm порты с использованием id security-group (enpo2auu0766h45ivobi)

```
1 yc vpc security-group update-rules enpo2auu0766h45ivobi --add-rule
  direction=ingress,protocol=tcp,port=2377,v4-cidrs=[10.128.0.0/24] --add-rule
  direction=ingress,protocol=tcp,port=7946,v4-cidrs=[10.128.0.0/24] --add-rule
  direction=ingress,protocol=udp,port=7946,v4-cidrs=[10.128.0.0/24] --add-rule
  direction=ingress,protocol=udp,port=4789,v4-cidrs=[10.128.0.0/24]
```

```
vboxuser@KVA:~/shvird-example-python$ yc vpc security-group update-rules enpo2auu0766h45ivobi --add-rule direction=ingress,protocol=tcp,port=2377,v4-cidrs=[10.128.0.0/24] --add-rule direction=ingress,protocol=tcp,port=7946,v4-cidrs=[10.128.0.0/24] --add-rule direction=ingress,protocol=udp,port=7946,v4-cidrs=[10.128.0.0/24] --add-rule direction=ingress,protocol=udp,port=4789,v4-cidrs=[10.128.0.0/24]
id: enpo2auu0766h45ivobi
folder_id: blgrsjmidldjs2rmkgd0
created_at: "2025-03-11T08:34:41Z"
name: default-sg-enp0sh2esfudgng27th4
description: Default security group for network
network_id: enp0sh2esfudgng27th4
status: ACTIVE
rules:
- id: enp5fi7s92g0l4oicv2h
  direction: INGRESS
  protocol_name: ANY
  protocol_number: "-1"
  cidr_blocks:
    v4_cidr_blocks:
      - 0.0.0.0/0
- id: enp5ft7trpt9elajaib7
  direction: EGRESS
  protocol_name: ANY
  protocol_number: "-1"
  cidr_blocks:
    v4_cidr_blocks:
      - 0.0.0.0/0
- id: enp4hpm8a5elj5gept3b
  direction: INGRESS
  ports:
    from_port: "2377"
    to_port: "2377"
  protocol_name: TCP
  protocol_number: "6"
  cidr_blocks:
    v4_cidr_blocks:
      - 10.128.0.0/24
- id: enp7ml4eu4qddbcgurq2
  direction: INGRESS
  ports:
    from_port: "7946"
    to_port: "7946"
  protocol_name: TCP
  protocol_number: "6"
  cidr_blocks:
    v4_cidr_blocks:
```

```
- 10.128.0.0/24
- id: enp2ajp5t3bf640sanhg
  direction: INGRESS
  ports:
    from_port: "7946"
    to_port: "7946"
  protocol_name: UDP
  protocol_number: "17"
  cidr_blocks:
    v4_cidr_blocks:
      - 10.128.0.0/24
- id: enptosi22i4fkm4rpd2a
  direction: INGRESS
  ports:
    from_port: "4789"
    to_port: "4789"
  protocol_name: UDP
  protocol_number: "17"
  cidr_blocks:
    v4_cidr_blocks:
      - 10.128.0.0/24
default_for_network: true
```

Инициализировал Swarm на Manager-ноде:

1	<code>ssh kva@93.77.176.108</code>	# подключился снаружи
2	<code>docker swarm init --advertise-addr 10.128.0.3</code>	# инициировал ноду

```
kva@f4m40nbf976l6nmvfsir:~$ docker swarm init --advertise-addr 10.128.0.3
Swarm initialized: current node (lxnqdyfm4i8dteypxw5236e4s) is now a manager.
```

To add a worker to this swarm, run the following command:

```
docker swarm join --token SWMTKN-1-2hddsuauxhd63behqke9v8hmt9y1a2rybx7jbq21e19g6s8qte-7r9bx4ypfvmpfua3x3zae2phn 10.128.0.3:2377
```

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

Подключил Worker-1 и Worker-2

```

1  ssh kva@130.193.51.191                                     # worker
1
2  docker swarm join --token SWMTKN-1-
  2hddsuauxhd63behqke9v8hmt9y1a2rybx7jbq21e19g6s8qte-7r9bx4ypfvmpfua3x3zae2phn
  10.128.0.3:2377
3
4  ssh kva@62.84.115.133                                       # worker
5
5  docker swarm join --token SWMTKN-1-
  2hddsuauxhd63behqke9v8hmt9y1a2rybx7jbq21e19g6s8qte-7r9bx4ypfvmpfua3x3zae2phn
  10.128.0.3:2377

```

```

kva@fhm3hlrjakk4dihvjcls:~$ docker swarm join --token SWMTKN-1-2hddsuauxhd63behqke9v8hmt9y1a2rybx7jbq21e19g6s8qte-7r9bx4ypfvmpfua3x3zae2phn 10.128.0.3:2377
This node joined a swarm as a worker.

```

```

kva@fhm38l3ehdf0efk8edk8:~$ docker swarm join --token SWMTKN-1-2hddsuauxhd63behqke9v8hmt9y1a2rybx7jbq21e19g6s8qte-7r9bx4ypfvmpfua3x3zae2phn 10.128.0.3:2377
This node joined a swarm as a worker.

```

Проверил кластер на manager-ноде:

```

1  ssh kva@93.77.176.108
2  docker node ls

```

```

kva@fhm40nbf976l6nmvfsir:~$ docker node ls

```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
iqrjdj2354zgoylgulxkxrmdz4	fhm3hlrjakk4dihvjcls	Ready	Active		29.4.2
xijj08vkvw0e2puod3poklsqw	fhm38l3ehdf0efk8edk8	Ready	Active		29.4.2
lxnqdyfm4i8dteypxw5236e4s *	fhm40nbf976l6nmvfsir	Ready	Active	Leader	29.4.2

Для deploy python-fork из прошлого ДЗ создал новый yaml конфигурационный файл **docker-new.yml**:

```
1  version: "3.8"
2
3  services:
4    web:
5      image: turistosartur/python-app:latest
6      environment:                                # из файла .env
7        - DB_HOST=db
8        - DB_NAME=virttd
9        - DB_USER=app
10       - DB_PASSWORD=QwErTy1234
11     deploy:
12       replicas: 2
13       restart_policy:
14         condition: on-failure
15       update_config:
16         parallelism: 1
17         delay: 10s
18     networks:
19       - backend
20
21  db:
```

```
22 image: mysql:8
23 environment:                                # из файла .env
24   - MYSQL_ROOT_PASSWORD=YtReWq4321
25   - MYSQL_DATABASE=virttd
26   - MYSQL_USER=app
27   - MYSQL_PASSWORD=QwErTy1234
28 volumes:
29   - db-data:/var/lib/mysql
30 deploy:
31   replicas: 1
32   placement:
33     constraints:
34       - node.role == manager
35 networks:
36   - backend
37
38 reverse-proxy:
39   image: haproxy:2.4
40   ports:
41     - "8080:8080"
42   configs:
43     - source: haproxy_cfg
44       target: /usr/local/etc/haproxy/haproxy.cfg
45   deploy:
```



```
46     replicas: 1
47     networks:
48         - backend
49
50     ingress-proxy:
51         image: nginx:latest
52         ports:
53             - "80:80"
54         configs:
55             - source: nginx_default
56               target: /etc/nginx/conf.d/default.conf
57             - source: nginx_main
58               target: /etc/nginx/nginx.conf
59         deploy:
60             replicas: 1
61         networks:
62             - backend
63
64     volumes:
65         db-data:
66
67     networks:
68         backend:
69             driver: overlay
```

```
70
71 configs:
72     haproxy_cfg:
73         file: ./haproxy/reverse/haproxy.cfg
74     nginx_default:
75         file: ./nginx/ingress/default.conf
76     nginx_main:
77         file: ./nginx/ingress/nginx.conf
```

1. Убрал `network_mode: host`, так как Swarm не поддерживает `host` режим. Вместо этого:

- Nginx слушает порт `80` внутри контейнера
- Swarm пробрасывает `80:80` на **все ноды** (routing mesh)

2. Volumes заменил на Configs, так как Swarm использует **Docker Configs** для конфигурационных файлов. Swarm **зашифрует** конфиг и доставит в контейнер:

```
1 configs:
2     haproxy_cfg:
3         file: ./haproxy/reverse/haproxy.cfg
4     nginx_default:
5         file: ./nginx/ingress/default.conf
6     nginx_main:
7         file: ./nginx/ingress/nginx.conf
```

3. Открыл порты наружу

```
1 - ingress-proxy:
2   image: nginx:latest
3   ports:
4     - "80:80"      # (доступен с любой ноды)
5 - reverse-proxy:
6   image: haproxy:2.4
7   ports:
8     - "8080:8080" # (доступен с любой ноды)
```

4. **Placement constraints:** прокси и БД запускаются **только на manager-ноде** (чтобы конфиги были доступны).

```
1 deploy:
2   replicas: 1
3   placement:
4     constraints:
5       - node.role == manager
```

Скопировал конфиги и файл на manager-ноду:

```
1 scp docker-new.yml kva@93.77.176.108:~/docker-new.yml
2
```

```
3  ssh kva@93.77.176.108 "mkdir -p ~/haproxy/reverse ~/nginx/ingress"
4
5  scp ./haproxy/reverse/haproxy.cfg kva@93.77.176.108:~/haproxy/reverse/
6  scp ./nginx/ingress/default.conf kva@93.77.176.108:~/nginx/ingress/
7  scp ./nginx/ingress/nginx.conf kva@93.77.176.108:~/nginx/ingress/
```

Поправил: `nano ~/haproxy/reverse/haproxy.cfg`

```
1  global
2      maxconn 1000
3
4  defaults
5      mode http
6      timeout connect 5000ms
7      timeout client 50000ms
8      timeout server 50000ms
9
10 frontend http_front
11     bind *:8080
12     default_backend http_back
13
14 backend http_back
15     balance roundrobin
16     server-template web 1-10 web:5000 check resolvers docker init-addr none
```

```
17
18 resolvers docker
19     nameserver dns1 127.0.0.11:53
20     resolve_retries 3
21     timeout resolve 1s
22     timeout retry 1s
23     hold valid 10s
```

Другой конфиг исправил: `nano ~/nginx/ingress/default.conf`

```
1  server {
2      listen 80;
3      server_name _;
4
5      access_log /var/log/nginx/access.log combined;
6
7      location / {
8          proxy_pass http://reverse-proxy:8080;
9
10         proxy_set_header Host $host;
11         proxy_set_header X-Real-IP $remote_addr;
12         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
13         proxy_set_header X-Forwarded-Proto $scheme;
14     }
```

```
15 }
```

Комментарии

1. `server-template web 1-10 web:5000`

- `web` - имя сервиса в Swarm (из `docker-new.yml`)
- `1-10` - HAProxy будет искать от 1 до 10 реплик этого сервиса
- `web:5000` - HAProxy запрашивает у DNS Swarm адреса всех контейнеров сервиса `web` . Swarm отдаст IP всех реплик: `10.0.1.5` , `10.0.1.8` и т.д.

2. `resolvers docker` - блок настроек **DNS-резолвера**.

- `nameserver dns1 127.0.0.11:53` - встроенный DNS-сервер Swarm, который запускает DNS на **каждой ноде** по адресу `127.0.0.11`
- `resolve_retries 3` - если DNS не ответил, попробовать 3 раза
- `timeout resolve 1s` - ждать ответа DNS максимум 1 секунду
- `hold valid 10s` - кешировать ответ DNS на 10 секунд (чтобы не спрашивать каждую миллисекунду)

3. `init-addr none` - при старте HAProxy **не паникует**, если DNS ещё не знает, где контейнеры (они могут стартовать позже).

По HAProxy убрал фиксированный ip адрес

По nginx default.conf:

1. `listen 80` Nginx слушает **стандартный HTTP-порт 80** внутри контейнера. В `docker-new.yml` прописал:
`ingress-proxy: ports: - "80:80"`. Swarm **автоматически** пробросит порт `80` на **все ноды кластера (Ingress Routing Mesh)**. С любой ноды (manager, worker-1, worker-2) по порту `80` Swarm **автоматически** перенаправит запрос к контейнеру Nginx (даже если он на другой ноде).
2. `proxy_pass http://reverse-proxy:8080`
 - `reverse-proxy` - имя сервиса HAProxy в `docker-new.yml`
 - Swarm резолвит это имя в **IP-адрес контейнера HAProxy** через встроенный DNS (`127.0.0.11`)

На Nginx убрали фиксированный адрес локалхост.

Запустил build образ на dockerhub под своим аккаунтом:

```
1 docker login
2 docker build -f Dockerfile.python -t turistosartur/python-app:latest .
3 docker push turistosartur/python-app:latest
4 docker pull turistosartur/python-app:latest #
   проверка
```

Деплой стека

На manager-ноде:

```
1 ssh kva@93.77.176.108
2 docker stack deploy -c docker-stack.yml python-app
```

```
kva@fhm40nbf976l6nmvfsir:~$ docker stack deploy -c docker-new.yml python-app
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network python-app_backend
Creating config python-app_nginx_default
Creating config python-app_nginx_main
Creating config python-app_haproxy_cfg
Creating service python-app_reverse-proxy
Creating service python-app_ingress-proxy
Creating service python-app_web
Creating service python-app_db
```

```
1 docker stack services python-app
```

```
kva@fhm40nbf976l6nmvfsir:~$ docker stack services python-app
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
pyghswcfrw5u	python-app_db	replicated	0/1	mysql:8	
74svw4xn3sve	python-app_ingress-proxy	replicated	0/1	nginx:latest	*:80->80/tcp
zrfce663zl76	python-app_reverse-proxy	replicated	0/1	haproxy:2.4	*:8080->8080/tcp
n44hprmihj8c	python-app_web	replicated	0/2	turistosartur/python-app:latest	


```

kva@fhm40nbf97616nmvfsir:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
8197a961534f   mysql:8        "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes  3306/tcp, 33060/tcp               python-app_db.1.yp6tp55t0bk5bch8cpugn27eb
69a8f9cbd4d6   nginx:latest   "/docker-entrypoint..." 4 minutes ago  Up 4 minutes  80/tcp                           python-app_ingress-proxy.1.zqglycfb5bnulepsdmd0r1gbl
kva@fhm40nbf97616nmvfsir:~$ docker exec -it 8197a961534f mysql -u app -pQwErTy1234 vird
mysql: [Warning] Using a password on the command line interface can be insecure.
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.4.9 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW TABLES;
+-----+
| Tables_in_vird |
+-----+
| requests       |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM requests LIMIT 10;
+-----+
| id | request_date      | request_ip |
+-----+
| 1  | 2026-05-02 20:30:28 | NULL      |
| 2  | 2026-05-02 20:31:30 | 10.0.0.4  |
| 3  | 2026-05-02 20:32:15 | 10.0.0.2  |
+-----+
3 rows in set (0.00 sec)

mysql>

```

При деплое возникли проблемы связности по причине некорректности первичного указания переменных окружения.

При подготовке решения использовал Claude 4.6 на стадии дебагинга и подготовки некоторых конфигов

Стенд удален из браузера